PDF

PDF EDITOR

PROJECT DOCUMENTATION

Functional scope, architecture, API, operations and deployment

Version 1.0 | Codebase assessment: 12 November 2026

LOCAL-FIRST DOCUMENT TOOLKIT

Document control

Field	Value
Project	PDF Editor / document converter
Document type	Project documentation
Prepared from	Current Flask, JavaScript, HTML/CSS and MySQL implementation
Scope	End-user tools, editor, API, persistence, administration, security, setup and limitations
Status	As implemented in the inspected codebase
Date	12 November 2026

Purpose

This document is the operational and technical reference for the PDF Editor project. It describes what the application does, how its parts interact, what each endpoint expects, how to deploy it, and which constraints must be understood before production use.

Audience

- Users and support staff who need a feature guide.
- Developers maintaining the Flask backend or browser editor.
- Administrators deploying the application and monitoring its activity.
- Testers validating file handling, editor output and access controls.

Executive summary

PDF Editor is a local-first web application that combines document conversion, common PDF utilities, a browser-based visual editor, and an administrative operations console. The browser sends files only to the Flask service running on the host; processing is performed with PyMuPDF, pypdf, ReportLab, pdf2docx and an office conversion executable.

The system is strongest for quick changes to text-based PDFs and for lossless structural operations. It does not include OCR, collaborative editing, cloud storage, or aggressive image recompression.

1. Product scope and feature map

The public interface exposes ten modes: the visual PDF editor plus nine focused file tools.

Feature	Input	Output	Implemented behavior
Edit PDF	PDF	PDF or DOCX	Edit existing lines; add styled text/images; highlight, cover, draw; rotate/select pages; undo/redo; zoom.
Word to PDF	DOC/DOCX	PDF	Converts locally through the configured office executable; supports batch processing in the UI.
PDF to Word	PDF	DOCX	Rebuilds editable Word content using pdf2docx; rejects encrypted and empty PDFs.
Merge	2+ PDFs	PDF	Appends files in the user-selected order.
Split / extract	PDF	PDF	Keeps pages specified as single numbers or ranges; typed order is preserved.
Compress	PDF	PDF	Compresses content streams and removes duplicate/orphan objects; images are not downsampled.
Protect	PDF	Encrypted PDF	Applies AES-256 user-password encryption; minimum password length is four characters.
Unlock	Encrypted PDF	PDF	Decrypts with the supplied current password; it does not recover unknown passwords.
Rotate	PDF	PDF	Rotates all pages or selected ranges by 90, 180 or 270 degrees.
Watermark	PDF	PDF	Adds Latin-text watermark to every page at diagonal, center or bottom position with selectable opacity.

Browser experience

- Drag-and-drop and file-picker input with type validation.
- Multi-file queues, ordering controls, per-file progress/error state, retry and clear actions.
- Individual downloads, ZIP download for batch results, and merge-after-conversion for Word files.
- Responsive public pages, light/dark theme support, feature explanations, use cases and FAQ content.
- Privacy messaging: no account requirement and processing on the machine hosting the service.

2. Visual editor

The editor creates a server-side session for an uploaded PDF and renders each page as a 2x PNG preview. Browser coordinates are stored as edit operations and applied to the source PDF only when the user exports.

Capability	Details
Existing text	Detects text lines and their spans, bounding box, baseline, size, color, font flags and sampled background. A changed line is redacted and redrawn, retaining span styles where practical.
Added text	Supports sans/serif/mono families, bold, italic, underline, size, color, left/center/right alignment, opacity, multi-line text and optional background fill.
Images	Add, paste or drop images; move and resize them. Existing PDF images can be identified and removed from a page.
Markup	Highlight or cover with rectangle, rounded rectangle or ellipse; freehand drawing with configurable color and width.
True cover	An opaque rectangular cover is applied as redaction so underlying text/image pixels do not reappear during copy or Word export.
Page controls	Rotate individual pages; select the pages retained in the exported document; zoom the working view.
History	Undo and redo are available in the browser, including Ctrl+Z and Ctrl+Y shortcuts.
Export	Save edited data as PDF, or render edits first and convert the result to DOCX.

Editor constraints

- Editing existing text is line-based, not full document reflow.
- Rotated pages do not expose existing text or image selection until upright.
- Scanned pages contain images, so existing text cannot be edited without OCR; users can still cover and add content.
- Font matching is approximate. Base PDF fonts are used for Latin-1; Arial/Segoe UI system fonts are attempted for wider Unicode coverage.
- Complex Word exports may require layout touch-ups.

3. User workflows

Edit and export a PDF

- Choose Edit PDF and open an unencrypted, non-empty PDF.
- Wait for page previews and selectable content metadata to load.
- Use Edit text, Add text, Highlight, Cover, Draw or Image; adjust contextual controls.
- Rotate pages if required, undo/redo changes, and choose which pages to retain.
- Select Save as PDF or Save as Word. The original file is not overwritten.

Run a utility

- Choose a tool from the header.
- Select or drop one or more supported files.
- Complete tool-specific options such as ranges, password, rotation angle or watermark settings.
- Run the operation and review each row for success, output size or a recoverable error.
- Download results individually, or download a ZIP when working with multiple files.

Page-range grammar

Ranges are one-based and comma-separated. Examples: **1**, **1-3**, or **1-3, 5, 8-10**. The server rejects malformed, descending, zero, empty and out-of-range selections. In the rotate tool, an empty range means all pages.

Expected error handling

Condition	Response
Wrong file extension or missing upload	HTTP 400 with a specific upload instruction.
Encrypted PDF sent to an unsupported tool	HTTP 400 directing the user to Unlock PDF.
Wrong unlock password	HTTP 400: wrong password.
Invalid page range / rotation / opacity	HTTP 400 with the invalid value or accepted range.
Office or PDF-to-Word conversion failure	HTTP 500 with a conversion error.
Expired editor session	HTTP 404; user must reopen the PDF.
Editor save failure	HTTP 500; source session remains available if not expired.

4. System architecture

The application follows a server-rendered, single-service architecture.

Presentation layer

- Jinja templates render public, policy, about, login and administration pages.
- app.js controls tool modes, queues, browser editor interactions, client validation and downloads.
- style.css and admin.css provide responsive public and administrator interfaces.

Application layer

- Flask defines public pages, file-processing APIs, editor-session APIs and superadmin routes.
- PyMuPDF handles rendering, text/image inspection, redaction, drawing and final editor output.
- pypdf handles merge, extraction, compression, encryption, decryption and page rotation.
- ReportLab produces watermark overlay pages; pdf2docx rebuilds DOCX output; lxml normalizes generated DOCX XML.
- A global lock serializes Word-to-PDF conversion to protect the office process.

Persistence and files

- MySQL stores editor-session metadata, operation logs, editor-save events and administrator accounts.
- Uploaded editor source PDFs are stored in the operating-system temp directory under pdf-editor-sessions.
- Utility outputs are generated in memory or temporary working folders and returned immediately.
- Editor sessions idle for six hours are removed opportunistically when a new editor file is opened.

Request flow

Browser -> Flask route -> validation -> processing library / temporary storage -> optional MySQL audit record -> streamed download. The WSGI application uses ProxyFix for one forwarding proxy hop.

5. HTTP API reference

Method and path	Form data	Success
POST /convert	file: DOC/DOCX	application/pdf
POST /pdf2word	file: PDF	DOCX MIME
POST /merge	file: repeated PDFs	application/pdf
POST /split	file; pages	application/pdf
POST /compress	file	application/pdf
POST /protect	file; password	AES-256 PDF
POST /unlock	file; password	decrypted PDF
POST /rotate	file; angle; optional pages	application/pdf
POST /watermark	file; text; position; opacity	application/pdf
POST /editor/open	file: PDF	JSON sid and page sizes
GET /editor/{sid}/page/{n}	path parameters	PNG preview
GET /editor/{sid}/lines/{n}	path parameters	JSON text-line metadata
GET /editor/{sid}/images/{n}	path parameters	JSON image references and boxes
POST /editor/{sid}/rotate/{n}	path parameters	JSON new page size
POST /editor/{sid}/save	edits JSON; keep JSON; format; img_* files	PDF or DOCX

Cross-cutting behavior

- CORS is currently set to Access-Control-Allow-Origin: * on every response.
- POST operations in the tool registry are timed and logged; password, edits and keep values are excluded from logged parameters.
- Downloads are streamed without changing the uploaded original.
- The API has no documented upload-size limit, authentication for public tools, or formal version prefix.

6. Data model and administration

Table	Purpose	Important fields
editor_sessions	Tracks active editing sessions.	sid, filename, page_count, size_bytes, created_at, updated_at
operations	Audits tool requests and outcomes.	tool, filename, input/output bytes, JSON params, status, error, duration, client IP
editor_saves	Records editor exports.	sid, format, edit_count, pages_kept, output_bytes
admins	Stores superadmin identities.	username, password_hash, role, created_at, last_login

Superadmin functions

- Authenticated dashboard with totals, success counts, input volume, average duration, last-24-hour activity, editor saves and a 14-day chart.
- Latest-operations view and paginated operations table.
- Filter by tool and status; search filename, IP address or error text.
- Export the current operation selection as CSV.
- Change the current administrator password with CSRF validation.
- Legacy /admin URLs redirect to /superadmin.

Authentication controls

- Passwords are stored with Werkzeug password hashes.
- Session cookies are HTTP-only and SameSite=Lax; persistent session duration is eight hours.
- Five failed login attempts per IP trigger an in-memory 15-minute throttle window.
- Password change requires the current password, eight to 128 characters, a changed value and confirmation.
- The first administrator is seeded from ADMIN_USER and ADMIN_PASSWORD; if no password is supplied, a random password is printed once.

7. Installation and configuration

Runtime prerequisites

- Python with packages: flask, pymupdf, pypdf, reportlab, pdf2docx, lxml, pymysql and cryptography.
- MySQL/MariaDB reachable from the Flask process.
- An office conversion executable named soffice on PATH for Word-to-PDF conversion (despite UI text referring to Microsoft Word).
- Windows fonts improve non-Latin editor text support; Arial and Segoe UI are probed when available.

Environment variables

Variable	Default / purpose
SECRET_KEY	If absent, generated once and persisted in .secret_key.
MYSQL_HOST	localhost
MYSQL_PORT	3306
MYSQL_USER	root
MYSQL_PASSWORD	empty
MYSQL_DATABASE	document_converter
ADMIN_USER	superadmin for the initial account
ADMIN_PASSWORD	Initial password; if absent, a generated value is printed

Startup procedure

- Create and activate a Python virtual environment.
- Install requirements from requirements.txt.
- Install/configure MySQL and ensure the configured user can create the database and tables.
- Install LibreOffice or another compatible soffice executable and add it to PATH.
- Set SECRET_KEY and initial admin credentials in production.
- Run python server.py. Startup initializes the database and seeds the first superadmin when necessary.
- Open the service URL shown by Flask; use /superadmin for administration.

Production deployment

Place the Flask application behind a production WSGI server and reverse proxy with TLS. The current ProxyFix configuration trusts one proxy hop, so the proxy topology must match. Configure request-size and timeout limits at both proxy and application-server layers.

8. Security, privacy and operational notes

Implemented safeguards

- PDF validation and explicit encrypted-file handling.
- AES-256 output encryption for Protect PDF.
- Editor session identifiers are random 32-character hex values and validated before path construction.
- Sensitive form fields are excluded from operation parameters.
- Administrator password hashing, CSRF check for password changes, login throttling and protected routes.
- Temporary editor data expires after six idle hours and orphaned files are also cleaned.

Production risks to address

- CORS is unrestricted. Restrict allowed origins before exposing the service beyond a trusted host.
- Public processing routes have no authentication, rate limit, upload-size limit or quota. Add controls appropriate to the deployment.
- The claim that files never leave the PC is true only when the service itself runs on that PC. A remotely hosted deployment sends files to that server.
- Operation logs store filenames, IP addresses and errors; define retention, access and privacy policies.
- Editor PDFs are temporarily stored unencrypted on disk. Protect the temp directory and consider encrypted storage for sensitive environments.
- The persisted .secret_key must be protected and backed up or supplied from a secret manager.
- In-memory login throttling resets on restart and is not shared across workers; use a shared rate limiter for scaled deployments.
- Error bodies may reveal backend details. Use structured, user-safe errors and private server logs in production.

Backup and maintenance

- Back up MySQL if audit history and administrator records must be retained.
- Do not back up transient editor-session files as durable documents.
- Monitor temp-disk usage, conversion duration, error rate and database growth.
- Patch Python packages, the office converter and MySQL regularly.

9. Limitations and test strategy

Known functional limitations

- No OCR for scanned PDFs.
- No form-field editor, digital signature workflow, annotation import/export, collaborative editing or cloud storage.
- Compression is lossless structural compression; image-heavy PDFs may shrink little.
- Watermarks accept Latin letters, numbers and common punctuation only.
- Editing is line-oriented and does not perform paragraph reflow.
- PDF-to-Word output fidelity depends on document complexity.
- The README is not currently a usable project guide; this document reflects the implementation itself.

Recommended acceptance tests

Area	Minimum validation
Conversions	DOC and DOCX success/failure; text, table and image PDF-to-DOCX; encrypted/empty rejection.
PDF utilities	Multi-file merge order; page grammar; compressed file opens; AES-256 protection; wrong/right unlock; selected-page rotation; all watermark positions.
Editor	Replace mixed-style text; add Unicode text; add/delete image; shapes; true redaction; freehand; rotate; page selection; undo/redo; PDF and DOCX output.
Security	Session-ID validation; expiration; admin auth/lockout; CSRF; password rules; safe logs; upload limits after implementation.
Compatibility	Chrome/Edge/Firefox; light/dark themes; phone/tablet/desktop layouts; representative PDF readers and Word versions.
Performance	Large page counts, image-heavy inputs, concurrent conversion requests, temp-disk pressure and database latency.

Release checklist

- Automated tests pass and representative output files are opened in independent viewers.
- Secrets, database credentials, TLS, CORS, upload limits and rate limits are configured.
- Admin password is changed and dashboard/export access is verified.
- Temporary storage and log-retention policies are documented.
- Backup, monitoring and recovery procedures are exercised.

10. Codebase map

Path	Responsibility
server.py	Flask routes, PDF operations, editor processing, administration and startup.
app.js	Tool-mode UI, queues, validation, downloads and visual editor state.
style.css	Public website and editor styles.
admin.css	Superadmin interface styles.
db.py	MySQL schema, sessions, audit operations, saves and admin queries.
templates/index.html	Main tool/editor surface and feature content.
templates/partials/*	Shared head, header and footer.
templates/admin*.html	Login, dashboard, latest operations, operations list and password pages.
templates/about.html	About page.
templates/privacy.html / terms.html	Policy pages.
requirements.txt	Python runtime dependencies.
server-upload.md	Existing server-upload notes/commands.

Maintenance guidance

- Keep UI tool definitions, server TOOLS logging keys and API routes synchronized.
- When adding a PDF operation, validate uploads through the shared read_pdf path and return consistent 400/500 errors.
- Exclude secrets and large payloads from operation logs.
- Preserve the editor coordinate transformation and page derotation logic when changing previews.
- Test both PDF and DOCX paths when modifying editor shapes, drawing or image handling.
- Update this documentation whenever routes, data tables, environment variables or user-visible capabilities change.

End of document

This reference was generated from the implementation present in the project workspace on 12 November 2026. Behavior should be revalidated after dependency or deployment changes.